If we don't specify a string value or we give `lock` a value of `true`, the database's default exclusive lock is obtained (normally this will be `"for update"`). We can often eliminate the need for this kind of locking using transactions (discussed starting in *Transactions*).

Databases do more than simply find and reliably retrieve data; they also do a bit of data reduction analysis. Rails provides access to these methods too.

Getting Column Statistics

Rails has the ability to perform statistics on the values in a column. For example, given a table of products, we can calculate the following:

```
average = Product.average(:price)  # average product price
max     = Product.maximum(:price)
min     = Product.minimum(:price)
total   = Product.sum(:price)
number  = Product.count
```

These all correspond to aggregate functions in the underlying database, but they work in a database-independent manner.

As before, methods can be combined:

```
Order.where("amount > 20").minimum(:amount)
```

These functions aggregate values. By default, they return a single result, producing, for example, the minimum order amount for orders meeting some condition. However, if you include the `group` method, the functions instead produce a series of results, one result for each set of records where the grouping expression has the same value. For example, the following calculates the maximum sale amount for each state:

```
result = Order.group(:state).maximum(:amount)
puts result  #=> {"TX"=>12345, "NC"=>3456, ...}
```

This code returns an ordered hash. You index it using the grouping element (`"TX"`, `"NC"`, … in our example). You can also iterate over the entries in order